

Object Lifecycle in Java

Introduction

In Java, the object lifecycle refers to the various stages an object goes through during its existence—from creation to destruction. Understanding this lifecycle is crucial for efficient memory management and avoiding issues like memory leaks.

An object in Java typically follows this lifecycle: Creation → Usage → Garbage Collection → Destruction

- **Creation:** Allocated memory on the heap using `new`.
- **Usage:** Methods and fields of the object are used.
- **Garbage Collection:** When no references point to the object, it becomes eligible for GC.
- **Destruction:** JVM garbage collector destroys and reclaims the memory.

To understand this better, consider the following code:

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Demo {
```

```
public:
```

```
    void performTask() {
```

```
        cout << "Task performed" << endl;
```

```
}  
};
```

```
int main() {  
    Demo obj;  
    obj.performTask();  
    return 0;  
}
```

```
import java.util.*;  
  
class Demo {  
    void performTask() {  
        System.out.println("Task performed");  
    }  
}
```

```
class Main {  
    public static void main(String[] args) {  
        Demo obj = new Demo();
```

```
        obj.performTask();
    }
}
```

```
from typing import List
```

```
class Demo:
```

```
    def performTask(self):
```

```
        print("Task performed")
```

```
if __name__ == "__main__":
```

```
    obj = Demo()
```

```
    obj.performTask()
```

What happens in memory?

- `obj` (the reference variable) is stored in stack memory.
- `new Demo()` creates an object that is stored in the heap memory.
- After the execution of `main()`, the `obj` reference goes out of scope.
- Now, no reference points to the `Demo` object in heap memory.
- Hence, Java's Garbage Collector will identify it as unreachable and **destroy the object**, freeing up memory.

This illustrates the automatic memory management Java provides and forms the foundation of the object lifecycle.

Object Creation

Objects are created using the **new** keyword, which allocates memory on the heap.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Student {
```

```
public:
```

```
    string name;
```

```
    Student(string name) {
```

```
        this->name = name;
```

```
    }
```

```
};
```

```
int main() {
```

```
    Student s("Raj");
```

```
    return 0;  
}
```

```
import java.util.*;  
  
class Student {  
    String name;  
  
    Student(String name) {  
        this.name = name;  
    }  
}
```

```
class Main {  
    public static void main(String[] args) {  
        Student s = new Student("Raj");  
    }  
}
```

```
from typing import List

class Student:

    def __init__(self, name):

        self.name = name


if __name__ == "__main__":

    s = Student("Raj")
```

Here in the above code, the `Student` object is created using the constructor. The `new` keyword allocates memory for the object on the heap, and the reference variable `s` points to that memory location.

Reference Counting

Reference counting is a **memory management technique** used in computer science to track how many parts of a program are using a specific resource (like an object or memory block). In Object Oriented Programming, each object keeps track of the number of references pointing to it. When an object's reference count drops to zero, it is considered unreachable and can be deallocated.

In simpler words, an object is eligible for GC (garbage collection) when it is no longer referenced from any live thread.

Note that, Java does not actually use **reference counting** as its garbage

collection mechanism, but understanding reference counting can still help us conceptually understand when objects become eligible for garbage collection.

Why it is not used in Java?

Java uses reachability-based garbage collection instead of reference counting.

- It uses graph traversal algorithms like Mark-and-Sweep to find all objects that are reachable from active threads.
- Even if reference count is 1 (in theory), an object could still be unreachable (like in the case of cyclic references).
- Java's GC handles cyclic references, which reference counting fails to clean up.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class A {};
```

```
int main() {
```

```
    A* a1 = new A(); // Object created
```

```
    A* a2 = a1;    // Two pointers to the same object
```

```
    a1 = nullptr; // One pointer remains
```

```
    a2 = nullptr; // Now zero pointers - memory leak unless deleted
```

```
// Note: In C++ this does NOT automatically free memory.  
  
// You'd normally call `delete` to free the object.  
  
return 0;  
  
}
```

```
import java.util.*;  
  
class A {}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        A a1 = new A(); // Object created  
  
        A a2 = a1;      // Two references  
  
  
        a1 = null;      // One reference remains  
  
        a2 = null;      // Now zero references -> Eligible for GC  
  
    }  
  
}
```



```
from typing import List
```

```
class A:
```

```
    pass
```

```
if __name__ == "__main__":
```

```
    a1 = A()  # Object created
```

```
    a2 = a1   # Two references
```

```
    a1 = None # One reference remains
```

```
    a2 = None # Now zero references -> Eligible for GC
```

While Java doesn't actually **count references**, this example helps learners understand how and when objects become unreachable.

Garbage Collection in Java

Java uses automatic garbage collection. The garbage collector (GC) identifies unused objects and frees memory. It is important to understand that the Garbage Collector isn't a single unified concept, but has multiple

implementations. Let's understand the Garbage Collection process in Java.

Garbage Collection Process in Java

At a broad level, Java's garbage collection (GC) process operates in three primary stages: marking, sweeping, and compacting. Depending on the specific garbage collector (like G1, ZGC, etc.), these stages may include more granular sub-steps, but the core concepts remain consistent across implementations.

- **Mark Phase:** When a new object is created, the Java Virtual Machine (JVM) assigns it a marking flag, initially set to false (or 0). This bit serves as an indicator of whether the object is still reachable.

The GC starts by marking all reachable objects, beginning from the root set (like local variables, static fields, etc.). It traverses the object graph, marking each object it encounters.

- **Sweep Phase:** Once marking is complete, the GC proceeds to the sweeping phase. Here, it goes through the heap and looks for objects that **remain unmarked** — meaning they were not found during the marking phase. These are considered garbage, as nothing in the program can access them anymore.

Such unreachable objects are then deleted, and their memory is reclaimed, making room for future allocations.

- **Compaction Phase:** Over time, object deletion creates gaps in memory, leading to fragmentation. The compaction phase addresses this by reorganizing live (reachable) objects to eliminate those gaps.

This is done by moving objects closer together, which not only optimizes memory usage but also improves cache performance.

Note that not all garbage collectors perform compaction. For example, the G1 garbage collector does this in a more sophisticated way, focusing on regions rather than the entire heap.

GC Pauses: Stop-the-World Events

During garbage collection, there can be moments when the JVM must **pause application** execution to safely manage memory. These are known as Stop-the-World (STW) events.

Because heap memory is not inherently thread-safe, allowing normal application threads to run while the GC is modifying object references could cause unpredictable behavior or errors. To prevent this, the JVM may halt all or parts of the program temporarily until the GC finishes its current phase, especially marking and compacting.

Some advanced garbage collectors (like G1, ZGC, or Shenandoah) try to minimize pause times, but some amount of pausing is usually unavoidable.

`System.gc();`

The `System.gc()` method is a request to the JVM to perform garbage collection. However, it is not a guarantee that the GC will run immediately. The JVM may choose to ignore this request based on its own heuristics and optimizations.

It is generally not recommended to call `System.gc()` in production code, as it can lead to unpredictable performance and may interfere with the JVM's own garbage collection strategy.

Memory Leaks

A memory leak happens when a program allocates memory but fails to release it after it's no longer needed. Over time, these unused memory blocks accumulate, leading to reduced available memory and potentially causing the program (or even the whole system) to slow down or crash.

Even though Java has an automatic garbage collection, a memory leak can still happen if the code **unintentionally keeps references** to objects that are no longer needed. As a result, the garbage collector **cannot reclaim** that memory, leading to increased memory usage over time.

A memory leak in Java typically happens when:

- Objects are stored in a static field, collection, or long-lived object and are not removed when no longer needed.
- Listeners or callbacks are added but not unregistered.
- Caches or maps grow indefinitely without proper cleanup.

Consider the following example to understand it better:

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class MemoryLeakExample {
```

```
private:
```

```
    // Static vector holds references for the entire life of the program
```

```
    static vector<Object*> staticList;
```

```
public:
```

```
    void addToStaticList(Object* obj) {
```

```
        // Every object added here is never removed, so memory keeps  
growing
```

```
        staticList.push_back(obj);
```

```
    }
```

```
};
```

```
// Definition of static member
```

```
vector<Object*> MemoryLeakExample::staticList;
```

```
class Object {
```

```
    // Empty class to simulate Java's Object
```

```
};
```

```
int main() {
```

```
    MemoryLeakExample example;
```

```
    for (int i = 0; i < 1000000; i++) {
```

```
        // Adding new objects continuously without freeing them
```

```
        example.addToStaticList(new Object());
```

```
}
```

```
cout << "Objects added to static list." << endl;
```

```
// Note: This code leaks memory because objects are never deleted.
```

```
return 0;
```

```
}
```

```
import java.util.*;
```

```
import java.util.*;
```

```
class MemoryLeakExample {
```

```
    // Static list holds references for the entire life of the program
```

```
    private static List<Object> staticList = new ArrayList<>();
```

```
    public void addToStaticList(Object obj) {
```

```
        // Every object added here is never removed, so memory keeps  
growing
```

```
        staticList.add(obj);
```

```
    }
```

```
}
```

```
class Main {
```

```
    public static void main(String[] args) {
```

```
        MemoryLeakExample example = new MemoryLeakExample();
```

```
        for (int i = 0; i < 1000000; i++) {
```

```
            // Adding new objects continuously without freeing them
```

```
            example.addToStaticList(new Object());
```

```
        }
```

```
        System.out.println("Objects added to static list.");
```

```
    }
```

```
}
```

```
from typing import List
```

```
class MemoryLeakExample:
```

```
    # Static list holds references for the entire life of the program
```

```
    staticList = []
```

```

def addToStaticList(self, obj):

    # Every object added here is never removed, so memory keeps
    growing

    MemoryLeakExample.staticList.append(obj)


class Object:

    pass # Simulate Java's Object class


if __name__ == "__main__":

    example = MemoryLeakExample()

    for i in range(1000000):

        # Adding new objects continuously without freeing them

        example.addToStaticList(Object())

    print("Objects added to static list.")

```

Understanding

In this example, the `staticList` holds references to all objects added to it. Since the list is static, it lives for the entire duration of the program. As a result, even though the loop creates a million objects, they are never

eligible for garbage collection because they are still referenced in `staticList`. Over time, this can lead to a `java.lang.OutOfMemoryError`, especially in long-running applications like servers or GUI apps.

Cyclic Reference

A cyclic reference occurs when two or more objects reference each other directly or indirectly, forming a loop, so none of them can be garbage collected—even if they are no longer accessible from the rest of the program.

The following example shows the formation of cyclic reference in a program:

```
#include <bits/stdc++.h>

using namespace std;
```

```
class Node {
public:
    Node* next;

    Node() : next(nullptr) {}
};
```

```
int main() {
    Node* a = new Node();
```

```
Node* b = new Node();  
  
a->next = b;  
  
b->next = a; // Cyclic reference formed
```

```
// Note: No automatic garbage collection in C++.
```

```
// Deleting nodes in a cycle requires manual break of links.
```

```
return 0;
```

```
}
```

```
import java.util.*;
```

```
class Node {
```

```
    Node next;
```

```
}
```

```
public class CyclicDemo {
```

```
    public static void main(String[] args) {
```

```
        Node a = new Node();
```

```
        Node b = new Node();
```

```
        a.next = b;
```

```
        b.next = a; // Cyclic reference formed
    }
}
```

```
from typing import List
```

```
class Node:
```

```
    def __init__(self):
```

```
        self.next = None
```

```
if __name__ == "__main__":
```

```
    a = Node()
```

```
    b = Node()
```

```
    a.next = b
```

```
    b.next = a # Cyclic reference formed
```

Languages like C++ don't have built-in garbage collection. So, if two objects reference each other and no external pointer refers to them, they will never be freed — causing a memory leak. This is especially tricky when

using shared pointers (e.g., `std::shared_ptr` in C++), because they rely on reference counts, and cyclic references keep that count above zero.

Unlike some languages, cyclic references do **not** cause memory leaks in Java, thanks to the reachability algorithm. Java's garbage collector is smart enough to detect unreachable objects, even if they reference each other in a cycle.

Best Practices

Here are some key best practices for managing the object lifecycle efficiently:

- **Avoid Explicit Null Assignments:** Manually setting objects to null rarely helps GC and can reduce readability.
- **Avoid Memory Leaks:** This can be done by clearing listeners and callbacks. Avoid overly complex or deeply nested data structures that are hard to track and clean up, especially when storing long-lived references.
- **Minimize Scope of Variables:** The sooner a reference goes out of scope, the earlier it becomes eligible for GC.